

signus 변경분

바뀐 파일은 통째로 실었습니다 — 그 파일 쪽만 새로 쓰면 됩니다

기준점 2026-08-22 (b1e550e+수정중) → 2026-08-22 · b1e550e+수정중 | 2개 파일 · +50 / -13 줄 · 코드 7쪽 | 새 코드북 79쪽

바뀐 내용

- 출력 용어를 알아듣기 쉽게 바꿨다: 반송파→중심주파수, 심볼레이트→baud. analyze 는 '시간상 버스트 N개 — 그중 k번을 분석', survey 는 '주파수상 방사체 N개 — 시간상 버스트 수가 아님'으로 찍어 두 숫자(시간상 버스트 vs 주파수상 방사체)를 헷갈리지 않게 했다. 경고문도 풀어 썼다.
- sa.py: 버스트마다 판독 숫자줄이 추가됐다 — 변조 자동판정 라벨(bpsk/qpsk/8psk/fsk/chirp/lin/flat)과 $x^2 \cdot x^4 \cdot x^8$ 바늘의 주파수·돌출 dB, AM 봉우리(=baud)를 검출 코드와 함께 찍는다. 이 줄들을 받아치면 그림 없이도 변조·중심주파수·baud 가 숫자로 전해진다.

□ signus/cli.py	+13	-10	2쪽	코드북 45쪽
□ probes/sa.py	+37	-3	5쪽	코드북 76쪽

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · **붉은 점선** = 그 자리에서 몇 줄 사라짐 · **↵** = 윗줄에 붙는 이어짐 (원래 한 줄이니 붙여서 입력)

signus/cli.py +13 -10 · 코드북 45쪽 · 새로 쓸 줄 77-80, 83, 89, 92, 94, 96-97, 126-127, 129

```

1 1  """CLI: analyze FILE / survey / serve. 합성·채점(gen/dataset/sweep)은 개발기 전용 lab.py 가
2 2  단다 -- 격리망 장비에는 그 파일이 없어 명령 자체가 없다."""
3 3
4 4  import argparse
5 5  import importlib.util
6 6  import re
7 7  import sys
8 8  from zlib import crc32
9 9
10 10 from .pipeline import analyze_file, survey_file
11 11 from .sigio import sidecar_read
12 12
13 13 _DTYPE_CHOICES = ("i8", "u8", "i16", "u16", "f32", "f64")
14 14
15 15
16 16 # --- 손으로 옮기는 한 줄 -----
17 17 # 실신호 장비는 인터넷도 클립보드도 없다. 결과는 사람이 화면을 보고 받아쳐서 나온다. 그래서
18 18 # 이 블록은 반드시 여기(필사 대상)에 있어야 한다 -- tools/ 에 두면 그 도구부터 필사해야 하는
19 19 # 본말전도가 된다. 줄 끝의 검출 코드가 "옮기다 한 글자 틀림"을 잡는다.
20 20
21 21 _ALPHA = "0123456789abcdefghijklmnopqrstuvwxyz" # 0/o, 1/l/i 처럼 화면에서 헷갈리는 글자를 뺀 32자
22 22 _BRIEF_FLAGS = [{"eq", lambda d: d["eq"]["applied"]},
23 23                 ("al", lambda d: d["detected"]["alias_resolved"]),
24 24                 ("fb", lambda d: d["detected"]["baud_fallback"]),
25 25                 ("amb", lambda d: d["detected"]["carrier_ambiguous"]),
26 26                 ("pre", lambda d: "preamble" in d["detected"])]
27 27
28 28
29 29 def check_code(text: str) -> str:
30 30     """받아친 줄의 오타 검출 코드 (4글자 = 20비트). 공백은 '한 칸으로 줄이되 없애지는'
31 31     않는다 -- 전부 지우면 'fs1000000 16qam' 과 'fs10000001 6qam' (샘플레이트 10배!) 이 같은
32 32     코드가 되어, 값이 바뀌는 오타 32종이 조용히 통과했다. 대소문자와 줄 간격은 계속 무시한다."""
33 33     body = re.sub(r"\s+", " ", re.sub(r"#[0-9a-z]{4}\b", "", text.lower())).strip()
34 34     n = crc32(body.encode())
35 35     return "".join(_ALPHA[(n >> (5 * i)) & 31] for i in (3, 2, 1, 0))
36 36
37 37
38 38 def brief(doc: dict, mode: str) -> str:
39 39     """손으로 옮기는 한 줄 + 검출 코드. Result.to_json() 디렉터리에서 만든다 -- 객체
40 40     속성을 직접 포맷하면 to_json 이 이미 한 번 돌고 두 번 겹쳐 장비와 맥이 갈린다."""
41 41     head = f"sig2 {mode} fs{doc['fs']:.0f} {doc['fmt']}-{doc['dtype']}"
42 42     if mode == "sv":
43 43         lines = [f"{head} n{doc['n_emitters']}"]
44 44         for i, e in enumerate(doc["emitters"][:12]):
45 45             baud = f" bd{e['baud']:.0f}" if e.get("baud") else ""
46 46             lock = f" lk{e['lock']:.0f}" if e.get("lock") is not None else ""
47 47             lines.append(f"{i} fc{e['abs_fc']:.0f}{baud}{lock} {e.get('mod') or e['kind']}")
48 48         if len(doc["emitters"]) > 12:
49 49             lines.append(f"...{len(doc['emitters']) - 12}개 생략")
50 50     else:
51 51         d, q = doc["detected"], doc["quality"]
52 52         p = [head, d["mod"], f"fc{d['fc']:.0f}", f"bd{d['baud']:.0f}", f"lk{q['lock']:.0f}"]
53 53         if q["mer_db"] is not None:
54 54             p.append(f"mer{q['mer_db']:.1f}")
55 55         if d.get("h") is not None: # FSK 는 롤오프 대신 변조지수
56 56             p.append(f"h{d['h']:.2f}")
57 57         elif d["rolloff"] is not None:
58 58             p.append(f"rl{d['rolloff']:.2f}")
59 59         if len(doc["bursts"]) > 1:

```

```

60 60         p.append(f"b{doc['burst_idx'] + 1}/{len(doc['bursts'])}")
61 61         p += [f for f, get in _BRIEF_FLAGS if get(doc)]
62 62         lines = [" ".join(p)]
63 63         body = "\n".join(lines)
64 64         return f"{body} #{check_code(body)}"
65 65
66 66
67 67 # --- analyze / survey / serve -----
68 68
69 69 def _analyze(args: argparse.Namespace) -> int:
70 70     r = analyze_file(args.file, args.fs, args.fmt, args.dtype,
71 71                     "be" if args.be else None, args.bitrev or None, args.diff,
72 72                     None if args.burst is None else args.burst - 1, rf=args.rf)
73 73     j = r.to_json(views=False) # 한 줄 요약도 여기서 만든다 (반올림이 한 번만 걸리게)
74 74     d = j["detected"]
75 75     truth = (sidecar_read(args.file) or {}).get("truth") # display only, never detection
76 76     if len(r.bursts) > 1:
77 +         print(f"시간상 버스트 {len(r.bursts)}개 - 그중 {r.burst_idx + 1}번을 분석"
78 +               " (--burst N 으로 선택)")
79 +         print(f"변조      {d['mod']}") + (f" (정답 {truth['mod']})" if truth else "")
80 +         print(f"중심주파수 {d['fc']:.1f} Hz" + (f" (정답 {truth['fc']:.1f})" if truth else ""))
81 81         if d["rf_hz"] is not None:
82 82             print(f"실제 RF      {d['rf_hz'] / 1e6:.6f} MHz")
83 +         print(f"baud      {d['baud']:.1f} Hz" + (f" (정답 {truth['baud']:.1f})" if truth else ""))
84 84         fsk = r.family == "fsk"
85 85         tail = f"변조지수 h {d['h']:.2f}" if fsk else f"롤오프      {d['rolloff']:.2f}"
86 86         mer = "" if fsk else f"MER {r.mer_db:.1f} dB"
87 87         print(f"{tail} lock {r.lock:.1f}{mer}")
88 88         if r.eq_applied:
89 +             print("다중경로 보정(등화기) 적용"
90 90                   + (" (T/2 분수간격)" if r.eq_mode == "fse" else " (심볼간격)"))
91 91         if r.alias_resolved:
92 +             print("중심주파수 접힘(앨리어스) 보정: 후보 중 스펙트럼 무게중심에 맞는 것을 선택")
93 93         if r.baud_fallback:
94 +             print("baud 폴백: 스펙트럼에서 baud 선을 못 찾아 점유대역폭으로 추정 - baud 신뢰 낮음")
95 95         if r.carrier_ambiguous:
96 +             print("경고: 중심주파수가 접혀 있을 수 있음 - fs 에 비해 너무 높다"
97 +               " (fc 를 그대로 믿지 말 것)")
98 98
99 99         if args.save_iq:
100 100             r.save_iq(args.save_iq)
101 101         if args.save_symbols:
102 102             r.save_symbols(args.save_symbols)
103 103         if args.save_bits:
104 104             r.save_bits(args.save_bits, args.packed)
105 105         if args.report:
106 106             r.save_report(args.report)
107 107         if args.brief: # 사람용 출력을 대체하지 않고 맨 끝에 한 줄 더 ---
108 108             print(brief(j, "an")) # 조기 return 이면 위 저장들이 조용히 무시된다
109 109         return 0
110 110
111 111 def _fhz(v: float) -> str:
112 112     a = abs(v)
113 113     if a >= 1e6:
114 114         return f"{v / 1e6:+.3f} MHz"
115 115     if a >= 1e3:
116 116         return f"{v / 1e3:+.2f} kHz"
117 117     return f"{v:+.0f} Hz"
118 118

```

```

117 119
118 120 def _survey(args: argparse.Namespace) -> int:
119 121     """Wideband survey: detect every emitter, demodulate the digital ones."""
120 122     s = survey_file(args.file, args.fs, args.fmt, args.dtype,
121 123         "be" if args.be else None, args.bitrev or None, args.diff, rf=args.rf)
122 124     rf0 = s.meta.rf_center
123 125     note = f" · RF 중심 {rf0 / 1e6:.3f} MHz" if rf0 is not None else ""
126 + print(f"주파수상 방사체 {len(s.emitters)}개 - 시간상 버스트 수가 아님"
127 +      f" (샘플레이트 {s.meta.fs:.0f} Hz{note})")
125 128     print(f"{'#':>2} {'실제 RF' if rf0 is not None else '중심주파수'}:>12} {'대역폭':>10}"
129 +      f" {'분류':>7} {'변조':>7} {'baud':>11} {'lock':>5}")
127 130     for i, e in enumerate(s.emitters):
128 131         r = e.result
129 132         mod = r.mod if r else "-"
130 133         baud = f"{r.baud:.0f}" if r else "-"
131 134         lock = f"{r.lock:.0f}" if r else "-"
132 135         kind = {"linear": "디지털", "fsk": "FSK", "analog": "아날로그",
133 136             "tone": "순수톤", "tooshort": "너무짧음", "error": "오류"}.get(e.kind, e.kind)
134 137         fc = e.abs_fc if rf0 is None else rf0 + e.abs_fc
135 138         print(f"{'i':>2} {_fhz(fc):>12} {_fhz(e.detection.bw):>10} {kind:>7}"
136 139             f" {mod:>7} {baud:>11} {lock:>5}")
137 140     if args.report:
138 141         import json
139 142         with open(args.report, "w") as fh:
140 143             json.dump(s.to_json(), fh, indent=1)
141 144     if args.brief:
142 145         print(brief(s.to_json(), "sv"))
143 146     return 0
144 147
145 148
146 149 def _read_args(p: argparse.ArgumentParser) -> None:
147 150     """Read-side sample-format overrides, shared by analyze/survey (sidecar/filename win)."""
148 151     p.add_argument("--fs", type=float)
149 152     p.add_argument("--fmt", choices=("iq", "real"))
150 153     p.add_argument("--dtype", choices=_DTYPE_CHOICES)
151 154     p.add_argument("--be", action="store_true", help="big-endian 샘플")
152 155     p.add_argument("--bitrev", action="store_true", help="바이트 내 비트 역순")
153 156     p.add_argument("--rf", type=float, help="RF 중심주파수 (Hz) - 실제 주파수로 보고")
154 157     p.add_argument("--brief", action="store_true",
155 158         help="손으로 옮길 한 줄 + 오타 검출 코드를 끝에 덧붙임")
156 159
157 160
158 161 def main(argv: list[str] | None = None) -> int:
159 162     ap = argparse.ArgumentParser(prog="signus")
160 163     sub = ap.add_subparsers(dest="cmd", required=True)
161 164
162 165     a = sub.add_parser("analyze", help="블라인드 복조 + 제원 탐지")
163 166     a.add_argument("file")
164 167     _read_args(a)
165 168     a.add_argument("--save-iq")
166 169     a.add_argument("--save-symbols")
167 170     a.add_argument("--save-bits")
168 171     a.add_argument("--packed", action="store_true")
169 172     a.add_argument("--report")
170 173     a.add_argument("--diff", action="store_true",
171 174         help="차동 디맵(D-BPSK/D-QPSK): 회전 모호성 없이 비트 복원")
172 175     a.add_argument("--burst", type=int, help="분석할 버스트 번호 (1부터; 기본 최강 버스트)")
173 176
174 177     sv = sub.add_parser("survey", help="광대역 캡처의 모든 신호 탐지 + 복조")
175 178     sv.add_argument("file")

```

```

176 179 _read_args(sv)
177 180 sv.add_argument("--diff", action="store_true", help="차동 디맵")
178 181 sv.add_argument("--report", help="JSON 리포트 저장 경로")
179 182
180 183 s = sub.add_parser("serve", help="웹 UI 서버")
181 184 s.add_argument("--host", default="127.0.0.1")
182 185 s.add_argument("--port", type=int, default=8000)
183 186
184 187 # 합성 생성·채점(gen/dataset/sweep)은 개발기 전용이다. 격리망 장비에는 lab.py 를
185 188 # 옮기지 않으므로 그 장비에서는 이 명령들이 아예 존재하지 않는다. try/except 로 삼키면
186 189 # 개발기에서 lab 내부의 진짜 임포트 오류까지 "명령 없음"으로 둔갑하니 파일 유무만 본다.
187 190 if importlib.util.find_spec("signus.lab"):
188 191     from . import lab
189 192     lab.add_commands(sub)
190 193
191 194 args = ap.parse_args(argv)
192 195 if args.cmd == "analyze":
193 196     return _analyze(args)
194 197 if args.cmd == "survey":
195 198     return _survey(args)
196 199 if args.cmd == "serve":
197 200     from .server import run
198 201     run(args.host, args.port)
199 202     return 0
200 203 return args.run(args)          # lab 이 단 명령 (gen / dataset / sweep)
201 204
202 205
203 206 if __name__ == "__main__":
204 207     sys.exit(main())

```

probes/sa.py +37 -3 · 코드북 76쪽 · 새로 쓸 줄 16, 18, 69-91, 97, 105-106, 111-115, 120, 186-188

```

1 1  """sa.py - 종합 관찰 프로브 (장비 필사용, signus/ 옆에 저장). 한 번에:
2 2  디버그 출력(포락선·버스트 표) + <캡처>.sa.png (스펙트로그램 + 검출 띠 + 버스트 번호 +
3 3  상위 버스트별 [PSD | x^2 | x^4 | x^8 | AM검파] 판독 행 - 바늘이 처음 서는 판이 변조).
4 4  python3 sa.py kat          # 자기검증: 표지 기대 출과 비교, sa-kat.png 생성
5 5  python3 sa.py <캡처파일> [K] # 판독할 버스트 수 K (기본 4, 에너지 상위부터)
6 6  x^m 판은 접힌 축(0~fs/2, 음수 주파수를 겹쳐 최대값) - 바늘 위치가 m·fc (mod fs) 다.
7 7  """
8 8  import struct
9 9  import sys
10 10 import zlib
11 11
12 12 import numpy as np
13 13 from scipy.signal import stft, welch
14 14
15 15 from signus import dsp, sigio
16 + from signus.chirp import is_chirp, sweeps_band
16 17 from signus.cli import check_code
18 + from signus.fsk import fsk_gate
17 19
18 20 GLYPH = {"0": (14, 17, 19, 21, 25, 17, 14), "1": (4, 12, 4, 4, 4, 4, 14),
19 21         "2": (14, 17, 1, 2, 4, 8, 31), "3": (31, 2, 4, 2, 1, 17, 14),
20 22         "4": (2, 6, 10, 18, 31, 2, 2), "5": (31, 16, 30, 1, 1, 17, 14),
21 23         "6": (6, 8, 16, 30, 17, 17, 14), "7": (31, 1, 2, 4, 8, 8, 8),
22 24         "8": (14, 17, 17, 14, 17, 17, 14), "9": (14, 17, 17, 15, 1, 2, 12)}
23 25
24 26
25 27 def draw_text(img, row, col, text, s=2):
26 28     for ch in text:
27 29         for r, bits in enumerate(GLYPH[ch]):

```

```

28 30         for c in range(5):
29 31             if bits >> (4 - c) & 1:
30 32                 img[row + r * s:row + r * s + s, col + c * s:col + c * s + s] = 1.0
31 33             col += 6 * s
32 34
33 35
34 36 def png_gray(img, path):
35 37     img = np.clip(img * 255, 0, 255).astype(np.uint8)
36 38     h, w = img.shape
37 39
38 40     def chunk(tag, data):
39 41         c = tag + data
40 42         return struct.pack(">I", len(data)) + c + struct.pack(">I", zlib.crc32(c))
41 43
42 44     raw = b"".join(b"\x00" + img[r].tobytes() for r in range(h))
43 45     with open(path, "wb") as fh:
44 46         fh.write(b"\x89PNG\r\n\x1a\n"
45 47             + chunk(b"IHDR", struct.pack(">IIBBBBB", w, h, 8, 0, 0, 0, 0))
46 48             + chunk(b>IDAT", zlib.compress(raw, 6)) + chunk(b"IEND", b""))
47 49
48 50
49 51 def panel(fvals, dbvals, wp=500, hp=120, grid=()):
50 52     """스펙트럼 곡선 한 판 - 바늘 보존을 위해 표시 칸마다 최대값 풀링."""
51 53     cols = np.clip((fvals / fvals.max() * (wp - 1)).astype(int), 0, wp - 1)
52 54     cur = np.full(wp, -1e9)
53 55     np.maximum.at(cur, cols, dbvals)
54 56     bad = cur < -1e8
55 57     cur[bad] = np.interp(np.flatnonzero(bad), np.flatnonzero(~bad), cur[~bad])
56 58     g_lo, g_hi = np.percentile(cur, 5) - 2, cur.max() + 2
57 59     img = np.zeros((hp, wp))
58 60     for gf in grid:
59 61         img[:,6, int(gf / fvals.max() * (wp - 1))] = 0.35
60 62     rows = ((hp - 1) * (1 - np.clip((cur - g_lo) / (g_hi - g_lo), 0, 1))).astype(int)
61 63     for cx in range(wp):
62 64         r0, r1 = (rows[cx], rows[cx - 1]) if cx else (rows[0], rows[0])
63 65         img[min(r0, r1):max(r0, r1) + 1, cx] = 1.0
64 66     return img
65 67
66 68
69 + def peak(fv, dbv, fmin=20.0):
70 +     """판 하나의 최대 바늘: (주파수 Hz, 중앙값 대비 돌출 dB) -- 숫자로 받아칠 수 있게."""
71 +     ok = fv >= fmin
72 +     i = int(np.argmax(dbv[ok]))
73 +     return round(float(fv[ok][i])), round(float(dbv[ok][i] - np.median(dbv[ok])))
74 +
75 +
76 + def verdict(z, fs, d2, d4, d8, dam):
77 +     """바늘 돌출률 버스트의 변조 부류를 판정 -- 문턱은 합성 배터리로 캘리브레이션
78 +     (bpsk d2 54 / qpsk d4 38 / 16qam d4 30 / 8psk d8 29 / fsk am 17, 광대역 잡음꼴 무바늘)."""
79 +     if is_chirp(z, fs) and sweeps_band(z, fs):
80 +         return "chirp" # 처프/LoRa 류 -- 성상도 복조 대상 아님
81 +     if fsk_gate(z, fs):
82 +         return "fsk"
83 +     if d2 >= 35 and d2 >= d4 + 5:
84 +         return "bpsk"
85 +     if d4 >= 28:
86 +         return "qpsk" # qpsk 또는 사각 QAM (4차에서 무너짐)
87 +     if d8 >= 24:
88 +         return "8psk"
89 +     return "lin" if dam >= 25 else "flat" # lin=선형(차수불명) / flat=바늘 없음(잡음꼴)

```

```

90 +
91 +
67 92 def burst_row(z, fs, label):
68 93     """한 버스트의 판독 행: [PSD |  $x^2$  |  $x^4$  |  $x^8$  | AM].  $x^m$  은 음수 주파수를 접는다."""
69 94     grid = tuple(fs / 2 * k / 5 for k in (1, 2, 3, 4))
70 95     fr, praw = welch(z.real, fs=fs, nperseg=min(2048, z.size))
71 96     ps = [panel(fr, 10 * np.log10(praw + 1e-18), grid=grid)]
97 +
73 98     nfft, nums = 1 << 15, []
74 99     f = np.fft.fftfreq(nfft, 1 / fs)
75 100     half = nfft // 2
76 101     for m in (2, 4, 8):
77 102         mag = np.abs(np.fft.fft((z ** m) * np.hanning(z.size), nfft))
78 103         fold = np.maximum(mag[:half], np.r_[mag[0], mag[1:half][::-1] * 0
79 104             + mag[half + 1:][::-1]])
105 +
106 +         ps.append(panel(f[:half], 20 * np.log10(fold + 1e-12), grid=grid))
107 +         pf, pd = peak(f[:half], 20 * np.log10(fold + 1e-12))
108 +         nums.append(f"p{m} f{pf} d{pd}")
80 107     u = np.abs(z) ** 2
81 108     u = u - u.mean()
82 109     su = np.abs(np.fft.rfft(u * np.hanning(u.size), nfft))
83 110     ps.append(panel(np.fft.rfftfreq(nfft, 1 / fs), 20 * np.log10(su + 1e-12), grid=grid))
111 +
112 +     af, ad = peak(np.fft.rfftfreq(nfft, 1 / fs), 20 * np.log10(su + 1e-12))
113 +     nums.append(f"am f{af} d{ad}")
114 +     lab = verdict(z, fs, int(nums[0].split("d")[1]), int(nums[1].split("d")[1]),
115 +         int(nums[2].split("d")[1]), ad)
116 +     nums.insert(0, lab)
84 116     div = np.full((120, 4), 0.5)
85 117     row = np.hstack(sum([p, div] for p in ps[:-1]), []) + [ps[-1]]
86 118     tag = np.zeros((120, 26))
87 119     draw_text(tag, 4, 4, label)
120 +
121 +     return np.hstack([tag, row]), " ".join(nums)
89 121
90 122
91 123 if len(sys.argv) < 2:
92 124     raise SystemExit("사용법: python3 sa.py kat | python3 sa.py <캡처파일> [버스트수 K]")
93 125 arg = sys.argv[1]
94 126 kmax = int(sys.argv[2]) if len(sys.argv) > 2 else 4
95 127 if arg == "kat":
96 128     fs, n = 10000.0, 80000
97 129     rng = np.random.default_rng(21) # 시드 고정 -- numpy 가 재현을 보장한다
98 130     x0 = 0.18 * rng.standard_normal(n)
99 131     t = np.arange(15000) / fs
100 132     for s0, fc, ph in ((12000, 550.0, 2), (50000, 550.0, 4)): # 앞=BPSK, 뒤=QPSK
101 133         up = np.zeros(15000, complex)
102 134         up[::34] = np.exp(2j * np.pi * rng.integers(0, ph, 442) / ph)
103 135         sym = np.convolve(up, np.hanning(68), "same") # 간이 펄스 성형
104 136         sym /= np.sqrt(np.mean(np.abs(sym) ** 2))
105 137         x0[s0:s0 + 15000] += (sym * np.exp(2j * np.pi * fc * t)).real * np.sqrt(2)
106 138         name = "sa-kat"
107 139 else:
108 140     x0, meta = sigio.read(arg)
109 141     fs = meta.fs
110 142     name = arg + ".sa"
111 143     xa = dsp.analytic(x0)
112 144     xa = xa - xa.mean()
113 145     n = xa.size
114 146     fb = dsp.find_bursts(xa, fs)
115 147     full = fb == [(0, n)]
116 148     pw = np.abs(xa) ** 2
117 149     lp = np.log10(np.maximum(np.convolve(pw, np.ones(64) / 64, "same"), 0) + 1e-20)

```

```

118 150 et = np.percentile(lp, 50)
119 151 ev = float(lp[lp >= et].mean() - lp[lp < et].mean())
120 152 print(f"n {n} fs {fs:.0f} 길이 {n / fs:.3f}s 포락선 분리(중앙 기준) {ev:.2f} decades")
121 153 print(f"find_bursts → {'통짜 [(0,n)] = 미검출' if full else str(len(fb)) + '개'}")
122 154 order = sorted(range(len(fb)), key=lambda i: -float(pw[fb[i][0]:fb[i][1]].sum()))
123 155 pick = sorted(order[:kmax]) if not full else []
124 156 for i, (s0, e0) in enumerate(fb if not full else []):
125 157     star = "★판독" if i in pick else ""
126 158     print(f" {i + 1:>2} 샘플 [{s0:>8}, {e0:>8}] {1000 * (e0 - s0) / fs:8.1f} ms"
127 159           f" 전력 {10 * np.log10(pw[s0:e0].mean() / (pw.mean() + 1e-30) + 1e-30):+5.1f} dB{star
128 160           }")
129 161
130 162 x_disp = x0.real if np.iscomplexobj(x0) else x0
131 163 _, _, z = stft(x_disp, fs=fs, window="hamming", nperseg=256, noverlap=128,
132 164               return_onesided=True, boundary=None, padded=False)
133 165 db = 10 * np.log10(np.abs(z) ** 2 + 1e-12)
134 166 lo = float(np.percentile(db, 25))
135 167 rg = float(max(10.0, min(35.0, np.percentile(db, 99.5) - lo)))
136 168 spec = np.clip((db - lo) / rg, 0, 1)[::-1]
137 169 kp = max(1, spec.shape[1] // 4000)
138 170 spec = spec[:, :spec.shape[1] // kp * kp].reshape(spec.shape[0], -1, kp).max(2)
139 171 pw_row = 26 + 5 * 500 + 4 * 4 # 판독 행 폭 -- 스트립을 여기에 맞춰 늘린다
140 172 fx = max(1, pw_row // spec.shape[1])
141 173 spec = np.repeat(spec, fx, axis=1)
142 174 ncol = spec.shape[1]
143 175 mark = np.zeros((18, ncol))
144 176 lane = np.zeros(ncol)
145 177 for i, (s0, e0) in enumerate(fb if not full else []):
146 178     c0 = (s0 // 128) // kp * fx
147 179     c1 = min(ncol - 1, (e0 // 128) // kp * fx)
148 180     lane[c0:c1 + 1] = 1.0
149 181     draw_text(mark, 2, min(c0 + 2, ncol - 14 * len(str(i + 1))), str(i + 1))
150 182 rows = [mark, spec, np.full((2, ncol), 0.35), np.tile(lane, (10, 1)),
151 183         np.full((6, ncol), 0.0)]
152 184 width = max(ncol, pw_row)
153 185 rows = [np.pad(r, ((0, 0), (0, width - r.shape[1]))) for r in rows]
154 186 for i in pick:
155 187     r, num = burst_row(xa[fb[i][0]:fb[i][1]], fs, str(i + 1))
156 188     ln = f"sa b{i + 1} {num}" # 바늘 주파수·돌출 -- 받아칠 판독 숫자줄
157 189     print(f"{ln} #{check_code(ln)}")
158 190     rows += [np.pad(r, ((0, 0), (0, width - r.shape[1])), np.full((6, width), 0.0))]
159 191 png_gray(np.vstack(rows), name + ".png")
160 192 line = (f"sa {'kat' if arg == 'kat' else 'cap'} n{n} f{fs:.0f} s{n / fs:.1f}"
161 193         f" fb{0 if full else len(fb)} ev{round(100 * ev)}")
162 194 print(f"{line} #{check_code(line)}")
163 195 print(f"{name}.png 저장 - 위: 스펙트로그램+검출 띠+버스트 번호, 아래: ★버스트별"
164 196       " [PSD|x2|x4|x8|AM] (x2 바늘=BPSK, x4=QPSK, x8=8PSK, AM 봉우리=심볼레이트)")

```